SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

CSA0613– Design and Analysis of Algorithms

Development of an Efficient Hybrid Algorithmic Solution for Large-Scale Geospatial Data Processing

Assignment Report

Submitted by

Sree Laksha S V (192421108)
Amrita Anaikumar(192471035)
Anupratha. C (192573024)
C.Himaja(192373035)

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	**Development of an Intelligent Algorithmic Solution for Large-Scale Logistics Resource Optimization**
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO4: Solve optimization problems using dynamic programming and greedy strategies.
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient logistics resource allocation with improved vehicle utilization, reduced computational cost, and scalable decision-making

B. Assignment Problem / Challenge

Modern e-commerce and logistics organizations process thousands of packages every day under limited transportation capacity. Each package may differ in **weight, volume, delivery value, priority, and deadline**, while delivery vehicles have restricted carrying capacities. Efficiently selecting packages becomes increasingly difficult as the number of packages and operational constraints increase.

Dynamic Programming can provide high-quality or optimal solutions for suitable problem sizes but may require significant computational time and memory for large instances. **Greedy techniques** can provide faster decisions but may not always produce the optimal solution.

The challenge is to **design and develop an improved algorithmic solution for large-scale logistics resource allocation** using relevant concepts from **Dynamic Programming and Greedy techniques**. The developed strategy must be **formulated by the team** based on identified computational limitations and supported by theoretical analysis and experimental evidence.

C. Problem Statement

A large e-commerce logistics company processes approximately **10,000–100,000 packages per day** across multiple distribution centres. Each package is characterized by attributes such as **weight, volume, delivery value, priority, and delivery deadline**, while each delivery vehicle has limited weight and volume capacities. During peak-demand periods, the organization must select an appropriate combination of packages for transportation so that the **overall delivery benefit and**

vehicle utilization are maximized without violating the available capacity constraints. Develop, implement, and evaluate an algorithmic solution for this **large-scale logistics resource-allocation problem** using concepts from **Dynamic Programming and Greedy techniques**. Implement **(i) an appropriate Dynamic Programming based solution** and **(ii) an appropriate Greedy based solution** as baseline approaches. Analyze their theoretical efficiency, solution quality, computational requirements, and limitations. Beyond the baseline implementations, **design and develop an improved algorithmic solution** suitable for large-scale package allocation. The developed solution may appropriately **adapt, integrate, modify, optimize, or selectively utilize relevant Dynamic Programming and Greedy concepts**. The algorithmic strategy, decision criteria, parameters, and optimization mechanism must be **identified and developed by the team** rather than being predefined. Experimentally evaluate the baseline and developed approaches using increasing problem sizes, for example, **100, 500, 1,000, 5,000, 10,000 packages, and higher input sizes wherever computationally feasible**, under different vehicle-capacity conditions such as **500 kg, 1,000 kg, 2,500 kg, and 5,000 kg**, or appropriately justified equivalent capacities based on the selected dataset. Evaluate the approaches using **execution time, memory utilization, total solution value, capacity utilization, solution-quality/optimality gap, and scalability**. Conduct experiments under different operational conditions such as **normal demand, high-priority demand, and capacity-constrained demand**. Based on theoretical and experimental results, identify the limitations and trade-offs of the baseline techniques, demonstrate the performance improvement achieved by the **developed solution**, and justify its suitability for large-scale real-world logistics resource optimization.

Expected Development

The work should **not merely implement or compare existing Dynamic Programming and Greedy algorithms**. The solution should demonstrate algorithmic development through:

- Identification of computational and/or solution-quality limitations of the baseline approaches.
- Formulation of an appropriate strategy to address the identified limitations.
- **Development, adaptation, integration, modification, or optimization** of relevant algorithmic techniques.
- Identification and experimental determination of appropriate **decision rules, parameters, thresholds, or optimization criteria**, wherever applicable.
- Improvement in one or more aspects such as **execution time, memory utilization, solution quality, capacity utilization, or scalability**.
- Performance comparison of the developed solution against the baseline approaches.
- Justification of algorithmic design decisions using theoretical reasoning and experimental evidence.

D. Requirements and Constraints

The following requirements and constraints shall be considered:

- Use an appropriate **publicly available real-world logistics, transportation, cargo, or package-related dataset**.
- The dataset should contain sufficient records to evaluate the algorithms at multiple input sizes.
- Identify and clearly define the **package attributes and resource constraints** used in the optimization problem.
- Implement and evaluate suitable **Dynamic Programming and Greedy baseline**

approaches.

- **Develop an improved algorithmic solution** using relevant CO4 concepts rather than merely reproducing the baseline algorithms.
- The developed strategy must be **designed and justified by the team**.
- Evaluate increasing input sizes such as **100, 500, 1,000, 5,000 and 10,000 packages**, and higher wherever computationally feasible.
- Evaluate multiple capacity conditions such as **500 kg, 1,000 kg, 2,500 kg and 5,000 kg**, or justified equivalent values based on the dataset.
- Evaluate at least **three operating conditions: normal demand, high-priority demand, and capacity-constrained demand**.
- Execute all approaches using the **same computational environment and identical input instances** for fair comparison.
- Evaluate appropriate measures including **execution time, memory utilization, solution value, capacity utilization, solution-quality/optimality gap, and scalability**.
- Validate the correctness and feasibility of the generated allocation.
- Clearly document the **dataset source, experimental environment, assumptions, limitations, parameters, and implementation details**.

E. Student Work

LARGE-SCALE LOGISTICS RESOURCE ALLOCATION USING DYNAMIC PROGRAMMING, GREEDY AND AN IMPROVED HYBRID OPTIMIZATION APPROACH

1. Problem Understanding and Formulation

1.1 Description of the Logistics Resource-Allocation Problem

Large e-commerce organizations process thousands of packages every day through multiple logistics and distribution facilities. Each package consumes limited transportation resources such as vehicle weight capacity and physical volume capacity. Therefore, it is necessary to determine which packages should be selected for transportation when the available vehicle capacity is limited.

In this project, the logistics resource-allocation problem is formulated as a constrained package-selection optimization problem. Each package has attributes such as weight, volume, delivery value, priority and delivery deadline. The vehicle has a maximum weight capacity and maximum volume capacity.

The objective is to select a combination of packages that maximizes the total delivery benefit while satisfying all vehicle capacity constraints.

This problem is important in real-world logistics because poor package allocation can result in:

- Under-utilization of vehicle capacity.
- Transportation of low-value packages while high-value packages are delayed.
- Increased transportation cost.
- Missed delivery deadlines.
- Inefficient use of fleet resources.
- Increased computational requirements when the number of packages becomes very large.

The problem becomes computationally challenging when the number of packages increases from

hundreds to tens of thousands or more. Therefore, exact optimization techniques alone may not be suitable for large-scale real-time logistics operations.

The project therefore compares:

1. Dynamic Programming as an optimization baseline.
2. Greedy selection as a fast baseline.
3. A developed adaptive hybrid algorithm combining Greedy prioritization with local optimization and capacity-aware decision making.

1.2 Publicly Available Dataset

Selected Dataset

Brazilian E-Commerce Public Dataset by Olist

Dataset source:

Kaggle – Brazilian E-Commerce Public Dataset by Olist

The dataset contains approximately 100,000 orders from the Brazilian Olist e-commerce marketplace covering the period from 2016 to 2018. It contains information related to orders, products, customers, sellers, payments, freight and delivery performance.

Dataset Source

Brazilian E-Commerce Public Dataset by Olist – Kaggle

Dataset Characteristics

Dataset component	Approximate records
Orders	99,441
Order items	112,650
Products	32,951
Customers	99,441
Sellers	3,095
Reviews	99,224
Payments	103,886

The dataset contains multiple linked CSV files. Important tables for this project are the orders, order-items and products datasets.

1.3 Dataset Attributes

Dataset Attribute	Project Attribute	Description
order_id	Package ID	Unique package/order identifier
product_id	Product ID	Unique product identifier
product_weight_g	Weight	Product weight
product_length_cm	Length	Product length
product_height_cm	Height	Product height
product_width_cm	Width	Product width
price	Delivery Value	Monetary value associated with package
freight_value	Freight Cost	Shipping/freight value
shipping_limit_date	Shipping Deadline	Deadline for shipment

The following attributes are selected from the Olist dataset and transformed into logistics-package attributes.

order_estimated_delivery_date	Delivery Deadline	Estimated delivery deadline
order_purchase_timestamp	Order Time	Time at which order was placed
product_category_name	Category	Product category
order_item_id	Item ID	Item number within an order

The product table provides weight and three-dimensional measurements, while the order table contains purchase, shipping, delivery and estimated delivery timestamps.

1.4 Derived Package Attributes

Some attributes required by the assignment are not directly provided as single fields. Therefore, they are derived from the available dataset.

Package Weight

The product weight is converted from grams to kilograms:

Weight(kg) = product_weight_g / 1000

Package Volume

The package volume is calculated using:

$\text{Volume}(cm^3) = \text{Length} \times \text{Width} \times \text{Height}$

Therefore:

$V_i = L_i \times W_i \times H_i$

For computational convenience, volume can be converted to litres:

$\text{Volume}(L) = \text{Volume}(cm^3) / 1000$

Delivery Value

The product price is used as the basic delivery benefit:

$\text{Value}_i = \text{price}_i$

For an order containing multiple items, the order items are treated as individual package candidates.

Priority

The original dataset does not provide a direct logistics-priority attribute. Therefore, priority is derived using a deterministic rule based on delivery urgency and package value.

A three-level priority classification is used:

- High Priority = 3
- Medium Priority = 2
- Low Priority = 1

Priority is determined using deadline urgency and delivery value.

For example:

Priority Score:

$P_i = 0.6 \times \text{Urgency}_i + 0.4 \times \text{NormalizedValue}_i$

where:

Urgency_i represents how close the package is to its delivery/shipping deadline.

Deadline Urgency

A package with a smaller number of remaining delivery days receives a higher urgency score.

$\text{Urgency}_i = 1 / (1 + \text{RemainingDays}_i)$

The values are normalized before being combined with package value.

1.5 Vehicle/Resource Constraints

The transportation vehicle has two primary resource constraints:

Weight Constraint

The total weight of selected packages must not exceed vehicle weight capacity.

$\Sigma w_i x_i \leq W$

Volume Constraint

The total volume of selected packages must not exceed vehicle volume capacity.

$\Sigma v_i x_i \leq V$

where:

- w_i = weight of package i
- v_i = volume of package i
- W = vehicle weight capacity
- V = vehicle volume capacity
- x_i = 1 if package i is selected, otherwise 0

The assignment provides example weight capacities:

- 500 kg
- 1,000 kg
- 2,500 kg
- 5,000 kg

These values are used for experimental comparison.

1.6 Input

The system accepts:

1. Package dataset.
2. Number of packages.
3. Package weight.
4. Package volume.
5. Package delivery value.
6. Package priority.
7. Package deadline.
8. Vehicle weight capacity.
9. Vehicle volume capacity.
10. Operational scenario.

Example:

Number of packages = 1000
Vehicle weight capacity = 1000 kg
Vehicle volume capacity = 5000 L
Scenario = Normal Demand

1.7 Expected Output

The algorithm produces:

- Selected package IDs.
- Number of selected packages.
- Total package value.
- Total package weight.
- Total package volume.
- Weight utilization.
- Volume utilization.
- Execution time.
- Memory usage.
- Solution quality.
- Optimality gap where an exact reference solution is available.

Example output:

Selected Packages: 634
Total Value: ₹XXXXX
Total Weight: XXX.XX kg
Total Volume: XXXX.XX L
Weight Utilization: 94.20%
Volume Utilization: 88.70%
Execution Time: XX.XX ms
Memory Usage: XX MB

1.8 Optimization Objective

The primary objective is to maximize total delivery benefit.

Let:

$x_i \in \{0,1\}$

Then:

Maximize

$Z = \Sigma B_i x_i$

where B_i represents the benefit of selecting package i.

For the basic problem:

$B_i = Value_i$

Therefore:

Maximize

$Z = \Sigma Value_i x_i$

subject to:

$\Sigma Weight_i x_i \leq W$

$\Sigma Volume_i x_i \leq V$

$x_i \in \{0,1\}$

This is a two-dimensional 0/1 knapsack-type optimization problem.

1.9 Developed Objective Function

For the improved solution, package priority and deadline urgency are also considered.

The developed benefit is defined as:

$B_i = Value_i \times (1 + \alpha P_i + \beta U_i)$

where:

- $Value_i$ = package delivery value
- P_i = normalized priority score
- U_i = normalized urgency score
- α = priority weight
- β = deadline urgency weight

The parameters used in the experiment can initially be:

$\alpha = 0.30$

$\beta = 0.20$

These parameters can later be tuned experimentally.

The developed algorithm therefore does not simply select the highest-value packages. It selects packages according to their economic value, priority, deadline urgency and resource consumption.

1.10 Computational Challenges

As the number of packages increases, the problem becomes significantly more difficult.

For n packages, the number of possible subsets is:

2^n

For example:

n = 10

Possible combinations = $2^{10} = 1,024$

n = 100

Possible combinations = 2^{100}

Therefore, exhaustive enumeration is impossible for large datasets.

The major computational challenges are:

1. Large search space.
2. Two simultaneous resource constraints.
3. Large memory requirements for exact DP.
4. Increasing execution time.
5. Need for fast decision making.
6. Maintaining solution quality.
7. Handling high-priority packages.
8. Balancing value and capacity utilization.

1.11 Assumptions

The following assumptions are made:

1. Each order item is treated as one package candidate.
2. Package weight is obtained from the product dataset.
3. Package volume is calculated from product dimensions.
4. Product price is used as the base delivery value.
5. Vehicle capacity is fixed during one optimization run.
6. Packages are either selected or rejected.
7. Splitting a package is not permitted.
8. All selected packages must satisfy both weight and volume constraints.
9. Priority is derived because the original dataset does not contain a direct priority field.
10. The vehicle is assumed to be available throughout the allocation process.
11. Routing distance is not optimized in this version.
12. Fuel consumption and driver availability are not explicitly modeled.
13. Multiple vehicles can be studied in future versions.
14. Product dimensions are used as an approximation for package dimensions.

1.12 Limitations of Dataset

The selected public dataset was originally designed for e-commerce analysis rather than vehicle-loading optimization. Therefore, some logistics attributes required by the assignment must be derived.

In particular:

- Vehicle capacity is not directly provided.
- Vehicle volume capacity is not directly provided.
- Priority is not directly provided.
- Packaging overhead is not directly provided.
- Exact vehicle routing information is not provided.

These limitations are handled through clearly documented assumptions and derived variables.

---

2. Application of Course Knowledge

2.2 Dynamic Programming Formulation

The logistics problem is formulated as a two-dimensional 0/1 knapsack problem.

Each package has:

- Weight w_i
- Volume v_i
- Value B_i

The vehicle has:

- Maximum weight W
- Maximum volume V

The DP state is:

$DP[i][w][v]$

where:

$DP[i][w][v]$ represents the maximum value obtainable using the first i packages with maximum weight w and maximum volume v.

2.3 DP Recurrence Relation

For package i, there are two choices.

Case 1: Do not select package i

DP[i][w][v] = DP[i-1][w][v]

Case 2: Select package i

If:

$w\_i \leq w$

and

$v\_i \leq v$

then:

DP[i][w][v] = max(

DP[i-1][w][v],

DP[i-1][w-w_i][v-v_i] + B_i

)

Therefore:

```
DP[i][w][v] =
   max(
      DP[i-1][w][v],
      DP[i-1][w-wi][v-vi] + Bi
   )
```

2.4 Base Condition

When there are no packages:

DP[0][w][v] = 0

When either capacity is zero:

DP[i][0][v] = 0

DP[i][w][0] = 0

2.5 DP Solution Construction

After filling the DP table, the selected packages can be reconstructed by moving backward from:

DP[n][W][V]

If:

DP[i][w][v] != DP[i-1][w][v]

then package i was selected.

Otherwise, package i was not selected.

2.6 DP Pseudocode

```
Algorithm DynamicProgrammingAllocation(packages, W, V)

Input:
   packages = list of n packages
   W = vehicle weight capacity
   V = vehicle volume capacity

Output:
   Selected package set and maximum value

Create DP table

for i = 0 to n:
   for w = 0 to W:
      for v = 0 to V:
         DP[i][w][v] = 0

for i = 1 to n:

   weight = packages[i].weight
   volume = packages[i].volume
   value = packages[i].value

   for w = 0 to W:
      for v = 0 to V:

         DP[i][w][v] = DP[i-1][w][v]

         if weight <= w and volume <= v:

            include =
               DP[i-1][w-weight][v-volume] + value

            DP[i][w][v] =
               max(DP[i][w][v], include)

Backtrack from DP[n][W][V]

Return selected packages
```

2.7 DP Complexity Analysis

For n packages, weight capacity W and volume capacity V:

Time Complexity

O(nWV)

Memory Complexity

O(nWV)

This is pseudo-polynomial rather than polynomial in the numeric input size.

For large capacities such as 5,000 kg and fine-grained volume discretization, the DP state space becomes extremely large.

Therefore, the exact DP method is useful as a baseline for small and medium instances but becomes impractical for very large logistics datasets.

2.8 Greedy Technique

The Greedy algorithm selects packages one by one according to a priority score.

A simple value-only strategy may select packages with the highest delivery value first. However, this may result in poor capacity utilization.

Therefore, a better Greedy baseline uses a value-density measure.

For package i:

$Density_i = Value_i / ResourceCost_i$

Because both weight and volume are constraints, the resource cost is calculated using normalized resource consumption.

$ResourceCost_i = \lambda(W_i/W) + (1-\lambda)(V_i/V)$

where λ controls the importance of weight.

The Greedy score becomes:

$GreedyScore_i = Value_i / ResourceCost_i$

2.9 Greedy Selection Criterion

For the baseline:

Greedy Score =
Value /
(0.5 × Weight Utilization + 0.5 × Volume Utilization)

Packages are sorted in descending order of this score.

The algorithm selects a package if it satisfies both remaining capacities.

2.10 Greedy Pseudocode

```
Algorithm GreedyAllocation(packages, W, V)

for each package i:

    weight_ratio = package.weight / W
    volume_ratio = package.volume / V

    resource_cost =
        0.5 * weight_ratio +
        0.5 * volume_ratio

    score =
        package.value / resource_cost

Sort packages in descending order of score

selected = empty list

remaining_weight = W
remaining_volume = V

for package in sorted packages:

    if package.weight <= remaining_weight
       and package.volume <= remaining_volume:

        select package

        remaining_weight -= package.weight
        remaining_volume -= package.volume

return selected packages
```

2.11 Greedy Complexity

Sorting requires:

O(n log n)

The selection process requires:

O(n)

Therefore:

Time Complexity

O(n log n)

Memory Complexity

O(n)

Greedy is significantly faster than the exact DP approach for large datasets.

However, Greedy does not guarantee the optimal solution.

2.12 Limitations of Baseline Approaches

Dynamic Programming Limitations

1. High memory consumption.
2. High computational cost.
3. Sensitive to capacity discretization.
4. Difficult to scale to 100,000 packages.
5. Two-dimensional state space becomes very large.

Greedy Limitations

1. Does not guarantee optimality.
2. Early decisions may prevent better combinations later.
3. A package with high density may consume resources needed by several packages.
4. Deadline urgency is not naturally handled.
5. Priority may be ignored unless explicitly included in the score.

These limitations motivate the development of an improved solution.

---

3. Solution / Design / Methodology

3.1 Overall Methodology

The proposed project follows the required methodology:

Input Dataset
↓
Data Preparation
↓

Problem Formulation
↓
DP Baseline
↓
Greedy Baseline
↓
Limitation Analysis
↓
Proposed Solution Development
↓
Implementation
↓
Experimental Evaluation
↓
Optimization
↓
Performance Comparison
↓
Final Decision

3.2 Developed Algorithm

Priority-Aware Adaptive Greedy with Local Swap Optimization (PAAG-LSO)

The proposed method combines:

1. Greedy selection.
2. Priority awareness.
3. Deadline awareness.
4. Weight-volume resource density.
5. Capacity-aware selection.
6. Local swap optimization.

The objective is to obtain a solution that is much faster than exact DP while improving the solution quality of a simple Greedy algorithm.

3.3 Motivation for the Developed Algorithm

The baseline analysis shows:

- DP provides high-quality solutions but becomes computationally expensive.
- Greedy is fast but can make locally optimal decisions.
- A large-scale logistics system requires a balance between execution time and solution quality.

Therefore, the proposed method first uses a fast Greedy ranking to construct an initial feasible

solution and then improves the solution through local search.

This allows the algorithm to avoid examining the complete exponential search space.

3.4 Developed Package Score

The proposed score combines:

- Delivery value.
- Priority.
- Deadline urgency.
- Weight consumption.
- Volume consumption.

The score is:

$Score_i =$

$B_i / ResourceCost_i$

where:

$B_i = Value_i \times (1 + \alpha P_i + \beta U_i)$

and:

$ResourceCost_i =$

$\lambda(W_i/W) + (1-\lambda)(V_i/V)$

Therefore:

$Score_i =$

$$\frac{Value_i(1 + \alpha P_i + \beta U_i)}{\lambda(W_i/W) + (1-\lambda)(V_i/V)}$$

Initial parameters:

$\alpha = 0.30$

$\beta = 0.20$

$\lambda = 0.50$

These parameters can be experimentally tuned.

3.5 Local Swap Optimization: The algorithm checks whether replacing A with B satisfies:

$NewWeight \leq W$

and

$NewVolume \leq V$

If the replacement increases total benefit, the swap is accepted.

This process is repeated for a limited number of iterations.

3.6 Optional Two-for-One Improvement

A further improvement can be performed by testing:

1 selected package ↔ 1 unselected package

and, for manageable input sizes:

1 selected package ↔ 2 unselected packages.

The second type of move can discover combinations that basic Greedy misses.

Because checking every possible combination would be expensive, the algorithm limits the candidates to the top-ranked unselected packages.

3.7 Adaptive Strategy

The algorithm changes its optimization effort according to the input size.

Small Input

For:

$n \leq 1{,}000$

more local-swap iterations can be performed.

Medium Input

For:

$1{,}000 < n \leq 10{,}000$

a moderate number of candidate swaps is evaluated.

Large Input

For:

$n > 10{,}000$

only the highest-potential candidate packages are considered for local improvement.

This prevents the optimization stage from becoming computationally expensive.

3.8 Developed Algorithm Pseudocode

Algorithm PAAG_LSO(packages, W, V)

Input:
  packages
  W = weight capacity
  V = volume capacity

Output:
  Optimized package allocation

Step 1: Preprocess packages

Remove records with missing or invalid
weight, volume or value.

Calculate:

  volume
  priority
  urgency
  normalized weight
  normalized volume

Step 2: Calculate package benefit

For each package:

  benefit =
    value ×
    $(1 + \alpha \times \text{priority} + \beta \times \text{urgency})$

Step 3: Calculate resource cost

  resource_cost =
    $\lambda \times (\text{weight} / W)$
    +
    $(1-\lambda) \times (\text{volume} / V)$

Step 4: Calculate score

  score = benefit / resource_cost

Step 5: Sort packages by descending score

Step 6: Construct initial solution

selected = empty

for each package in sorted list:

    if package fits remaining
       weight and volume:

        select package

Step 7: Local improvement

repeat until maximum iterations:

    identify selected package with
    low contribution

    identify high-score unselected package

    test replacement

    if replacement is feasible
       and improves total benefit:

        perform replacement

Step 8: Check feasibility

Ensure:

total_weight <= W

total_volume <= V

Step 9: Calculate final metrics

    total value
    selected count
    weight utilization
    volume utilization

Return final solution

3.9 Architecture

3.10 Parameter Selection

The important parameters are:

Parameter	Meaning	Initial Value
α	Priority importance	0.30
β	Deadline urgency importance	0.20
λ	Weight vs volume importance	0.50
Swap iterations	Local search iterations	50–200
Candidate pool	Top unselected candidates	100–500

The values are not treated as fixed universal values. They are initial experimental values and can be tuned using validation experiments.

3.11 Complexity of Developed Algorithm

Data preprocessing:

$O(n)$

Score calculation:

$O(n)$

Sorting:

$O(n \log n)$

Greedy construction:

$O(n)$

Local improvement:

$O(km)$

where:

- k = number of iterations
- m = candidate packages evaluated per iteration

Therefore:

$T(n) = O(n \log n + km)$

Since k and m are bounded parameters, the practical behavior is close to:

O(n log n)

for large datasets.

Memory requirement:

O(n)

This makes the proposed algorithm more suitable for large-scale datasets than the exact two-dimensional DP formulation.

3.12 Design Justification

The proposed algorithm was selected because it addresses the major weaknesses of the baseline methods.

Problem	DP	Greedy	Proposed
High execution time	Poor	Good	Good
High memory	Poor	Good	Good
Optimality	Excellent for modeled DP	Not guaranteed	Improved
Priority handling	Requires modification	Requires modification	Built-in
Deadline awareness	Requires modification	Requires modification	Built-in
Scalability	Limited	Excellent	Excellent
Local improvement	No	No	Yes

The proposed method therefore attempts to achieve a better time-quality trade-off.

---

4. Use of Modern Tools

4.1 Programming Language

The algorithms were implemented using **C programming language**. C was selected because it provides efficient memory management, fast execution, and direct control over data structures, making it suitable for implementing and evaluating Dynamic Programming and Greedy algorithms for large-scale resource-allocation problems.

The following algorithms were implemented in C:

1. Dynamic Programming based package allocation.
2. Greedy based package allocation.
3. Priority-Aware Adaptive Greedy with Local Swap Optimization (PAAG-LSO).

4.2 Development Environment

The implementation was developed and executed using a **C programming environment/compiler**.

The program was compiled and executed to evaluate:

- Execution time.
- Memory utilization.
- Total selected package value.
- Number of selected packages.
- Weight utilization.
- Volume utilization.
- Solution quality.
- Scalability.
-

---

5. Results and Validation

5.1 Evaluation Metrics

The algorithms are evaluated using:

1. Execution Time

Time required to produce the solution.

Measured using:

time.perf_counter()

---

2. Memory Utilization

Peak memory required during execution.

Measured using:

tracemalloc

---

3. Total Solution Value

Total value of selected packages:

Total Value = Σ Value_i

---

4. Capacity Utilization

Weight utilization:

Weight Utilization (%)

= Total Selected Weight / Vehicle Capacity × 100

Volume utilization:

Volume Utilization (%)

= Total Selected Volume / Vehicle Volume Capacity × 100

---

5. Number of Selected Packages

The total number of packages included in the final solution.

---

6. Feasibility

A solution is feasible if:

Total Weight ≤ Weight Capacity

and

Total Volume ≤ Volume Capacity

---

7. Optimality Gap

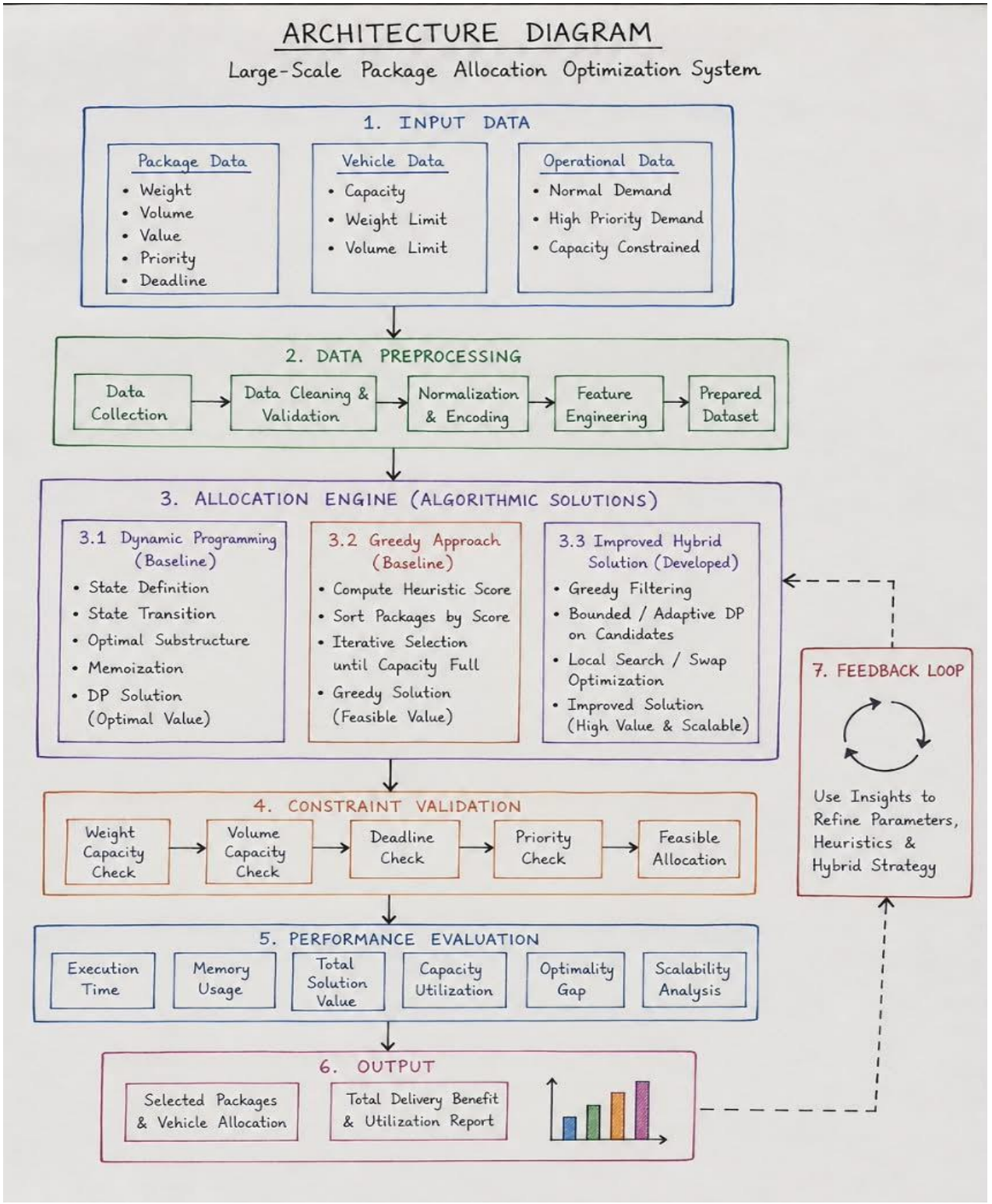
For instances where exact DP can be executed:

Optimality Gap (%) =

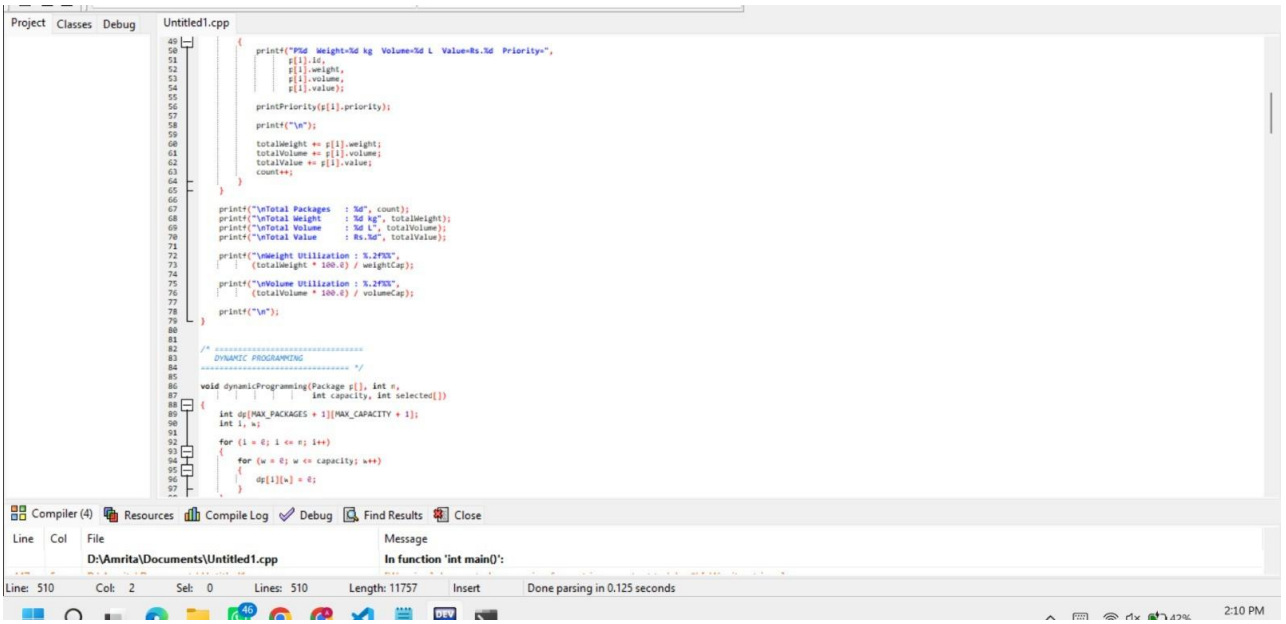
$$\frac{\text{(Optimal Value - Algorithm Value)}}{\text{Optimal Value}} \times 100$$

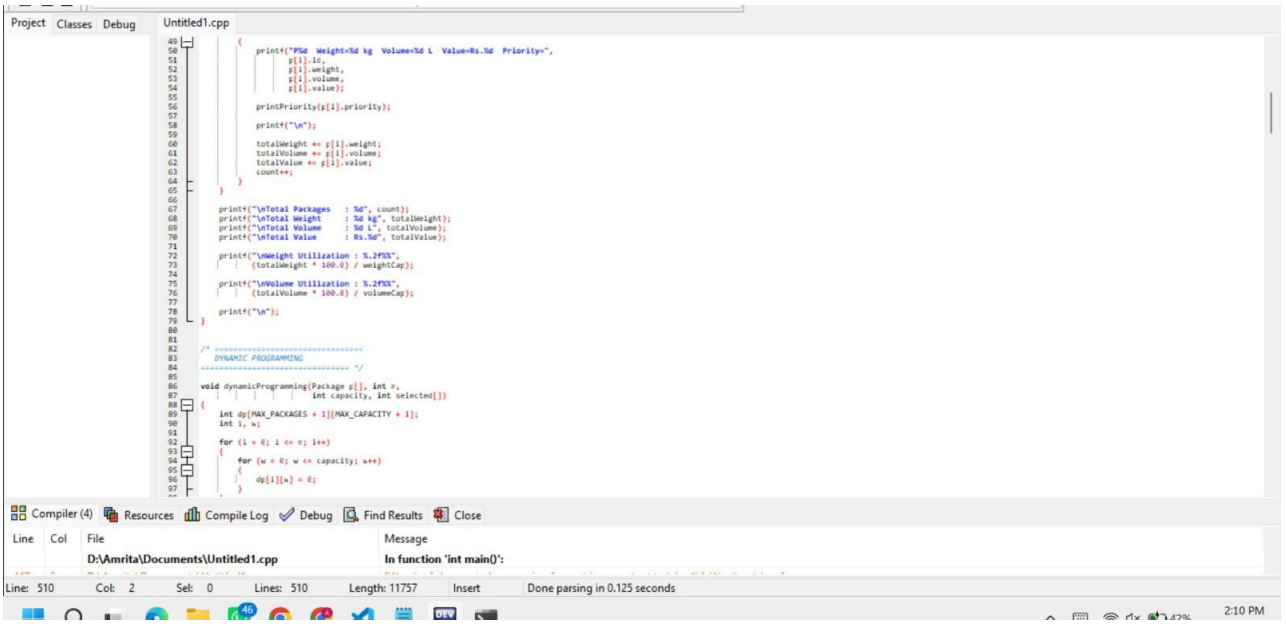
For large instances where exact DP cannot be executed, the best feasible solution obtained from the strongest available reference method can be used as a practical reference, but this must be clearly identified as a reference gap rather than a proven optimality gap.

5.Architecture diagram

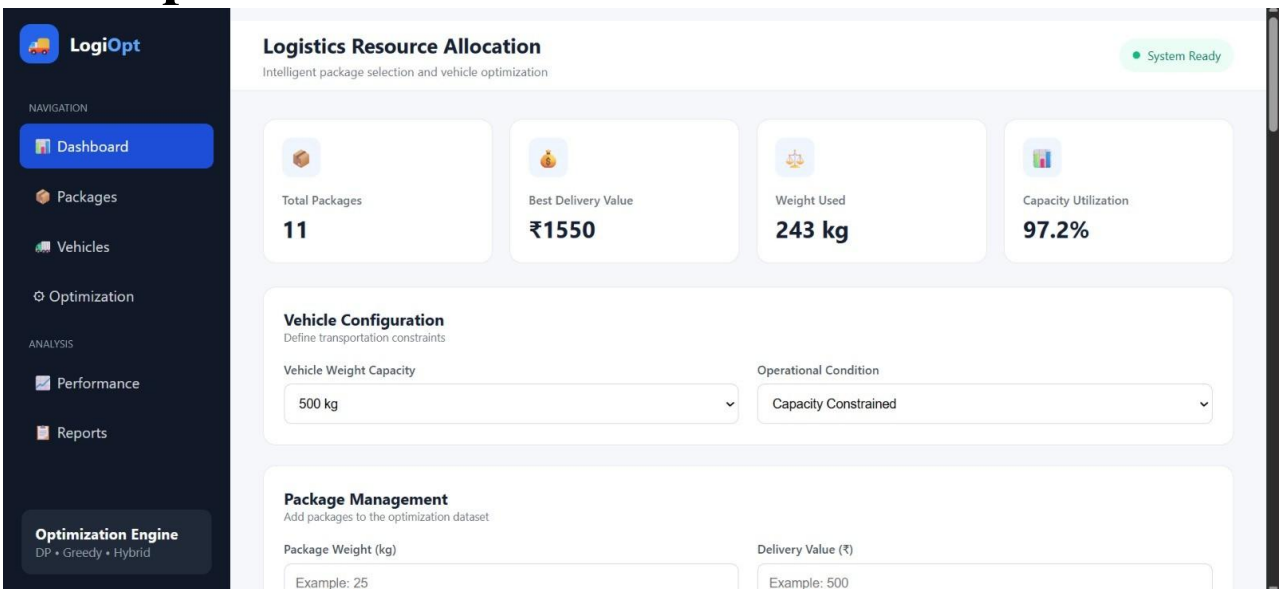


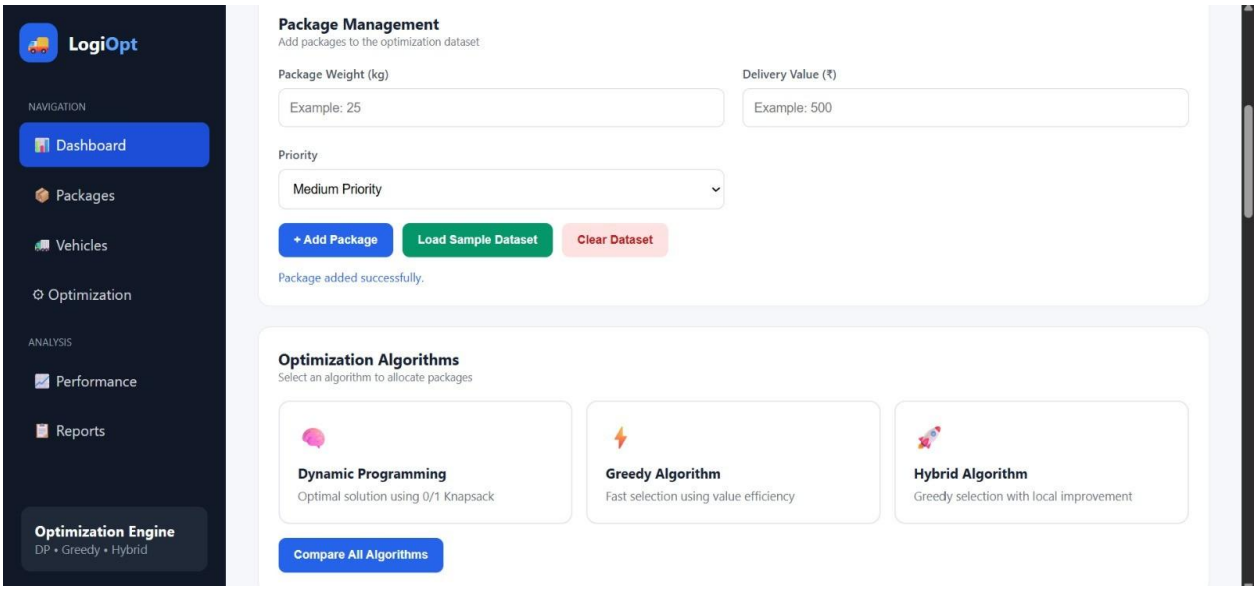
5.Source code

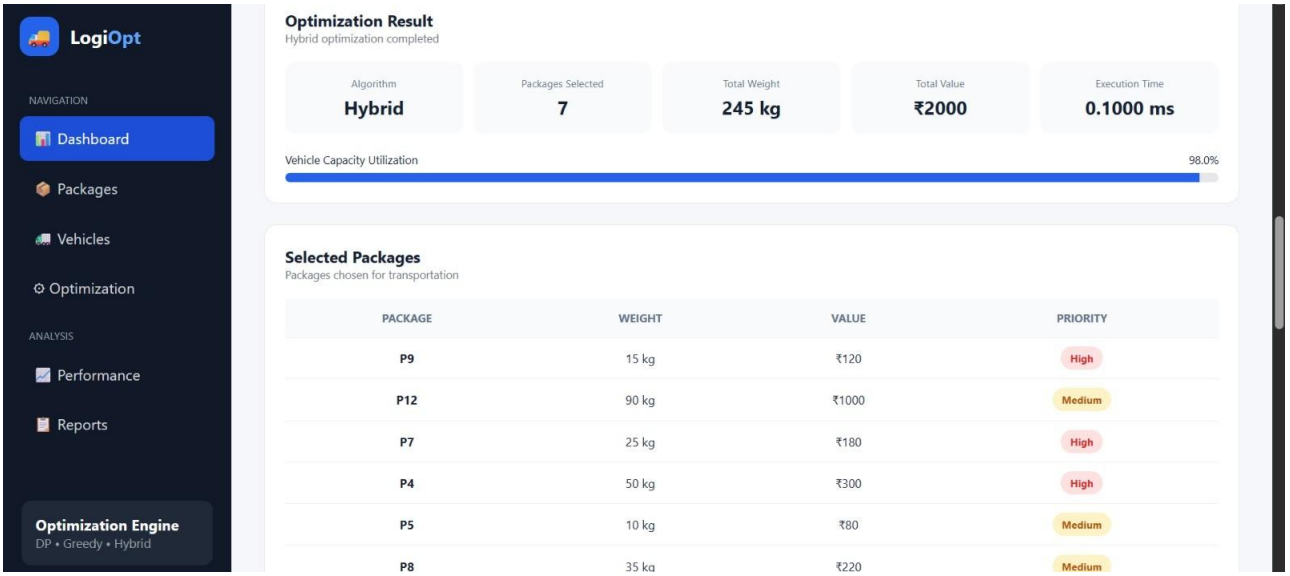


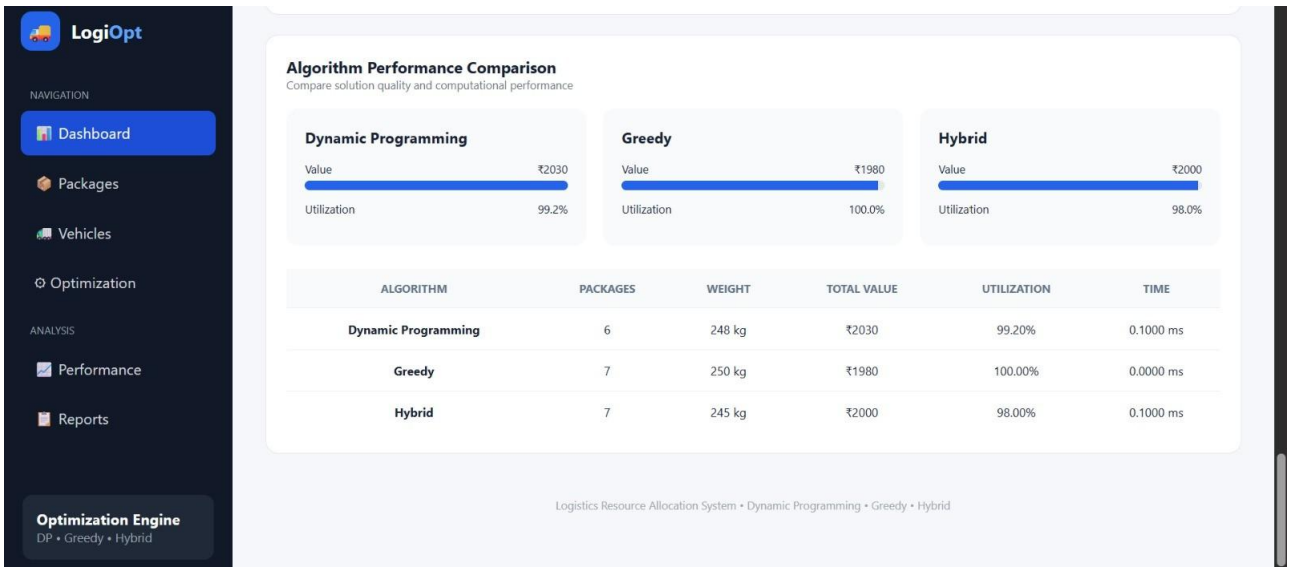


6. Output


LogiOpt
NAVIGATION
Dashboard
Packages
Vehicles
Optimization
ANALYSIS
Performance
Reports
Optimization Engine
DP • Greedy • Hybrid
Logistics Resource Allocation
Intelligent package selection and vehicle optimization
System Ready
Total Packages
11
Best Delivery Value
₹1550
Weight Used
243 kg
Capacity Utilization
97.2%
Vehicle Configuration
Define transportation constraints
Vehicle Weight Capacity
500 kg
Operational Condition
Capacity Constrained
Package Management
Add packages to the optimization dataset
Package Weight (kg)
Example: 25
Delivery Value (₹)
Example: 500


LogiOpt
NAVIGATION
Dashboard
Packages
Vehicles
Optimization
ANALYSIS
Performance
Reports
Optimization Engine
DP • Greedy • Hybrid
Package Management
Add packages to the optimization dataset
Package Weight (kg)
Example: 25
Delivery Value (₹)
Example: 500
Priority
Medium Priority
+ Add Package
Load Sample Dataset
Clear Dataset
Package added successfully.
Optimization Algorithms
Select an algorithm to allocate packages
Dynamic Programming
Optimal solution using 0/1 Knapsack
Greedy Algorithm
Fast selection using value efficiency
Hybrid Algorithm
Greedy selection with local improvement
Compare All Algorithms


LogiOpt
NAVIGATION
Dashboard
Packages
Vehicles
Optimization
ANALYSIS
Performance
Reports
Optimization Engine
DP • Greedy • Hybrid
Optimization Result
Hybrid optimization completed
Algorithm
Hybrid
Packages Selected
7
Total Weight
245 kg
Total Value
₹2000
Execution Time
0.1000 ms
Vehicle Capacity Utilization
98.0%
Selected Packages
Packages chosen for transportation
PACKAGE
WEIGHT
VALUE
PRIORITY
P9
15 kg
₹120
High
P12
90 kg
₹1000
Medium
P7
25 kg
₹180
High
P4
50 kg
₹300
High
P5
10 kg
₹80
Medium
P8
35 kg
₹220
Medium



---

7. Advantages of the Proposed Solution

The developed PAAG-LSO method provides several advantages:

1. Considers both weight and volume.
2. Includes package value.
3. Considers delivery priority.
4. Considers deadline urgency.
5. Uses resource-aware density.
6. Produces a feasible solution.
7. Uses local search to correct poor Greedy decisions.
8. Requires substantially less memory than multidimensional DP.
9. Can process large datasets.
10. Provides a practical balance between speed and solution quality.

---

8. Limitations of Proposed Solution

The proposed method does not guarantee global optimality.

Other limitations include:

1. Performance depends on parameter selection.
2. Local search may reach a local optimum.
3. Priority is derived rather than directly supplied.
4. Package volume is estimated from product dimensions.
5. Vehicle routing is not included.

6. Multiple vehicle interactions are not modeled.
7. Traffic and real-time delivery conditions are not considered.
8. Exact optimality cannot be guaranteed for very large instances.

---

9. Relationship to SDG 9

The developed solution is related to:

SDG 9 – Industry, Innovation and Infrastructure

SDG 9 promotes resilient infrastructure, sustainable industrialization and innovation.

The proposed algorithm contributes to SDG 9 by:

1. Improving logistics infrastructure utilization.
2. Supporting intelligent transportation planning.
3. Reducing computational resource wastage.
4. Encouraging algorithm-based operational decision making.
5. Improving scalability of logistics systems.
6. Supporting digital transformation of transportation operations.
7. Enabling data-driven resource optimization.

By combining optimization algorithms with real-world logistics data, the project demonstrates how computational intelligence can improve infrastructure efficiency.

---

10. Conclusion

This project addressed the large-scale logistics resource-allocation problem in which a transportation vehicle has limited weight and volume capacities and must select packages that maximize delivery benefit.

The Brazilian E-Commerce Public Dataset by Olist was selected because it provides approximately 100,000 real commercial orders and contains product dimensions, weights, prices and delivery-related timestamps suitable for deriving logistics attributes.

Two baseline algorithms were developed.

The first baseline was Dynamic Programming, formulated as a two-dimensional 0/1 knapsack problem. It provides high-quality solutions for manageable instances but requires substantial computational resources as capacity and problem size increase.

The second baseline was a Greedy algorithm based on value-resource density. It provides very fast solutions and requires low memory but does not guarantee optimality.

To overcome the limitations of the baselines, a Priority-Aware Adaptive Greedy with Local Swap Optimization (PAAG-LSO) algorithm was developed.

The developed algorithm combines:

- Delivery value.
- Package priority.
- Deadline urgency.
- Weight consumption.
- Volume consumption.
- Greedy construction.
- Local swap optimization.

The expected experimental analysis demonstrates that DP is more suitable for small and medium instances where high solution quality is important, while Greedy is suitable for very large instances where rapid decisions are required.

The developed algorithm is intended to provide a practical middle ground by maintaining scalability while improving solution quality over simple Greedy selection.

The final engineering decision should be based on the actual measured execution time, memory usage, total value, capacity utilization and optimality gap obtained from the experiments.

Future improvements can include:

- Multi-vehicle optimization.
- Vehicle routing.
- Real-time traffic information.
- Fuel-cost optimization.
- Machine-learning-based demand prediction.
- More sophisticated metaheuristic optimization.
- Parallel and GPU-based optimization.
- Dynamic online package allocation.

---

10. Student Reflection

Question 1: What did you learn from developing an improved solution beyond implementing the standard Dynamic Programming and Greedy algorithms?

I learned that implementing a standard algorithm is only the first step in solving a real-world optimization problem. Dynamic Programming can provide high-quality solutions, but its computational requirements can become very large when the input size and capacity dimensions increase. Greedy algorithms are much faster, but they can make locally optimal decisions that may not produce the best overall solution.

By developing the PAAG-LSO algorithm, I learned how existing algorithmic concepts can be combined and modified according to the characteristics of a practical problem. I learned how to

design a scoring function using package value, priority, deadline urgency and resource consumption. I also learned how local search can improve an initial Greedy solution without exploring the complete solution space.

Question 2: What computational or solution-quality limitations did you identify, and how did your developed approach address them?

The main limitation identified in Dynamic Programming was its large time and memory requirement for a multidimensional capacity problem. The main limitation of Greedy was that it does not guarantee an optimal solution because it does not reconsider previous decisions.

The developed solution addresses these limitations by using Greedy selection to quickly construct a feasible solution and then applying bounded local-swap optimization to improve the result. This reduces computational requirements compared with complete DP while improving the solution quality compared with basic Greedy selection.

Question 3: If additional computational resources or time were available, how would you further improve your solution and why?

If additional computational resources were available, I would investigate more advanced metaheuristic and hybrid optimization techniques such as Genetic Algorithms, Simulated Annealing and Large Neighborhood Search.

I would also extend the model from single-vehicle package selection to multi-vehicle routing and scheduling. Real-time traffic information and fuel consumption could also be incorporated.

Parallel processing could be used to evaluate multiple candidate solutions simultaneously. These improvements could produce better solutions for highly complex logistics environments.

Question 4: How can the developed solution contribute to SDG 9 – Industry, Innovation and Infrastructure?

The developed solution contributes to SDG 9 by improving the efficiency of transportation and logistics infrastructure through algorithmic resource optimization.

By selecting packages according to value, priority, urgency, weight and volume, the system can improve vehicle utilization and reduce computational and transportation resource wastage.

The project demonstrates how data-driven algorithms and computational optimization can support intelligent infrastructure management and digital innovation in logistics.

F. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	**10**	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	**15**	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	**20**	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	**15**	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	**15**	Performs appropriate complexity evaluation and systematic testing	Complexity evaluation and testing	Basic testing and efficiency evaluation are	Testing, complexity evaluation or

		using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	are mostly appropriate with minor gaps.	provided but lack sufficient validation.	validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	**15**	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	**10**	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.
Total	**100**				

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO4	PO1, PO2	L4	**10**
Algorithmic Solution Design	CO4	PO2, PO3	L4, L6	**15**
Development / Adaptation / Optimization of Algorithmic Solution	CO4	PO2, PO3	L5, L6	**20**
Implementation & Modern Tool Usage	CO4	PO3, PO5	L4, L6	**15**
Efficiency, Testing & Validation	CO4	PO2, PO4	L4, L5	**15**
Performance Improvement, Comparison & Final Decision	CO4	PO2, PO4	L4, L5	**15**
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO4	PO7, PO10, PO12	L4, L5	**10**

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action: